

Blocco 10 - Gentle Introduction: Window function

Calcoli analitici senza perdere il dettaglio delle righe

Relational Databases & SQL

Materiale aggiuntivo

Perché questo materiale

Una introduzione progressiva alla sintassi e al metodo del blocco 10.

Problema di partenza

Situazione concreta

Vogliamo ranking, totali progressivi e confronti con la riga precedente mantenendo le righe originali.

Ponte con i blocchi precedenti

GROUP BY comprime righe; le window function aggiungono calcoli mantenendo la granularità di partenza.

Idea guida

Una finestra è il gruppo di righe che una funzione può guardare mentre calcola il valore della riga corrente.

- ▶ prima si scrive la domanda in linguaggio naturale;
- ▶ poi si decide il soggetto della query;
- ▶ poi si dichiara la granularità del risultato;
- ▶ solo alla fine si rifinisce la sintassi.

Sintassi essenziale

La sintassi è utile solo se rappresenta una decisione chiara sul significato delle righe.

Schema sintattico

```
funzione(...) OVER (  
    PARTITION BY gruppo  
    ORDER BY ordinamento  
) AS nuova_colonna
```

Come leggere la sintassi

- ▶ `OVER` trasforma una funzione in funzione finestra.
- ▶ `PARTITION BY` definisce gruppi indipendenti.
- ▶ `ORDER BY` ordina le righe dentro la finestra.
- ▶ `row_number`, `rank`, `lag` sono funzioni analitiche comuni.
- ▶ La query conserva una riga per riga di input.

Dal linguaggio naturale a SQL

Una query è una frase controllabile: soggetto, condizioni, trasformazioni e risultato. Se una parte della frase resta ambigua, anche il codice diventa fragile.

- ▶ il soggetto decide da dove partire;
- ▶ le condizioni decidono cosa includere;
- ▶ le trasformazioni decidono cosa calcolare;
- ▶ l'ordinamento decide come leggere il risultato.

Esempi progressivi

Ogni esempio parte da una richiesta informale, passa per una specifica e arriva alla soluzione SQL.

Esempio 1: Ranking ordini per cliente

Domanda

Ranking ordini per cliente.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 1: implementazione

```
SELECT customer_id,  
       order_id,  
       order_date,  
       gross_revenue,  
       row_number() OVER (  
         PARTITION BY customer_id  
         ORDER BY order_date, order_id  
       ) AS order_seq  
FROM order_revenue  
ORDER BY customer_id, order_seq;
```

Esempio 2: Top prodotti per categoria

Domanda

Top prodotti per categoria.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 2: implementazione

```
SELECT *
FROM (
  SELECT p.category,
         p.product_name,
         sum(oi.quantity) AS units,
         rank() OVER (
           PARTITION BY p.category
           ORDER BY sum(oi.quantity) DESC
         ) AS category_rank
  FROM order_items AS oi
  JOIN products AS p ON p.product_id = oi.product_id
  GROUP BY p.category, p.product_name
) AS ranked
WHERE category_rank <= 3
ORDER BY category, category_rank;
```

Esempio 3: Ricavo progressivo per canale

Domanda

Ricavo progressivo per canale.

Controllo

Prima di leggere il codice, dichiarare popolazione, granularità e colonne finali attese.

Esempio 3: implementazione

```
SELECT channel,
       order_date,
       order_id,
       gross_revenue,
       sum(gross_revenue) OVER (
         PARTITION BY channel
         ORDER BY order_date, order_id
       ) AS running_revenue
FROM order_revenue
WHERE status NOT IN ('cancelled', 'refunded')
ORDER BY channel, order_date, order_id;
```

Esercizio guidato

Data una rappresentazione tabellare, costruire la query che risponde alla richiesta.

Rappresentazione tabellare

cliente	ordine	data	ricavo
1	1	2025-01-06	1418.25
1	15	2025-04-03	335.00
1	31	2025-06-12	1470.90

Richiesta

Data la tabella ordini, costruire la query che assegna a ogni ordine il numero progressivo dell'ordine per cliente.

Metodo

Evidenziare prima la granularità del risultato, poi scrivere la query.

- ▶ usare window function quando serve invece un riepilogo con GROUP BY;
- ▶ dimenticare ORDER BY nei calcoli progressivi;
- ▶ non distinguere row_number e rank;
- ▶ non specificare partizioni quando il confronto è per cliente o paese;
- ▶ interpretare il totale progressivo senza guardare l'ordinamento.

Checklist finale

- ❶ La domanda informale è stata tradotta in specifica?
- ❷ La granularità del risultato è dichiarata?
- ❸ I filtri e i collegamenti sono motivati?
- ❹ I nomi delle colonne finali sono leggibili?
- ❺ Esiste almeno una query di controllo?

- ▶ scrivere SQL significa rendere verificabile una domanda;
- ▶ ogni costrutto sintattico deve corrispondere a una scelta di modello;
- ▶ esempi piccoli e controlli intermedi evitano errori difficili da vedere.